

2018
SCON
Solutions



A번

PLAYERJINAH'S BOTTLEGROUNDS



PLAYERJINAH'S BOTTLEGROUND

- 유형: 기하, 수학
- 푸는 방법은 여러가지 (벡터, 기울기, CCW, ...)
- 여기서는 가장 간단한 기울기 등식을 이용해 해결
 - 나머지 해법이 궁금하다면 SCCC에 가입을 하시면 됩니다
- 세개의 점 $P1(x1, y1)$, $P2(x2, y2)$, $P3(x3, y3)$ 이라고 할 때,
- $P1-P2$, $P2-P3$ 를 잇는 선분의 기울기를 각각 $L12$, $L23$ 라고 해보자.
 - $L12 = (x1-x2) / (y1-y2) = dx12 / dy12$
 - $L23 = (x2-x3) / (y2-y3) = dx23 / dy23$
 - 세 점이 직선상에 위치하는 경우는 $L12 = L23$ 인 경우이다.
 - 즉 $dx12 / dy12 = dx23 / dy23$
 - $= dx12 * dy23 = dx23 * dy12$
- 고로 $(x1-x2) * (y2-y3) = (x2-x3) * (y1-y2)$ 인지만 보면 알 수 있음!

B번
저거 못타면 지각이야!



저거 못타면 지각이야!!

- 유형: 시뮬레이션, 구현
- 1차선으로 구성된 버스 정류장에서 버스들의 진입, 정차, 떠나는 상황들을 관리 해줘야 한다.
- 이를 적당히 시뮬레이션하는 코드를 작성할 수 있으면 해결 가능.

저거 못타면 지각이야!!

• 시뮬레이션하는 한 방법으로는 다음과 같은 방법을 사용

1. 정류장의 가장 뒤에 위치한 버스가 떠나지 않는다면, 진입하는 버스 입장에서는 그 다음에 위치할 수 밖에 없다. 즉, 가장 뒤에 위치한 버스가 떠났는지에 대해서만 궁금하다.
2. 가장 뒤에 위치한 버스가 떠나기 위해선 앞에 모든 버스가 떠났어야 했다.

=> 떠날 수 있는 버스들도 잠시 대기하다가 모두가 동시에 떠날 수 있으면 다같이 떠나도, 뒤에는 영향을 미치지 않는다.

=> 배열에 데이터를 담아두다가 모두가 동시에 떠날 수 있을 때, 데이터를 전부 제거

저거 못타면 지각이야!!

- 시간 관리의 경우 새로운 버스가 진입할 때마다 계산해주면 된다.

1. (현재 시간 - 이전 버스 진입시간)를 흐른 시간 T 로 정의한다.
2. $T > 0$ 인 경우, 해당 시간만큼 정류장에 위치한 버스들의 정차시간에 T 를 빼준다.
3. 정류장의 모든 버스가 나갈 수 있다면, 정류장을 비운다.
4. 만약, 정류장에 여석이 없을 경우, 정류장에 위치한 버스들 중 정차시간이 가장 큰 버스의 정차 시간을 T 로 재설정 한 후 2번부터 다시 진행한다.
5. 버스를 정류장에 넣는다.

- 다음과 같은 방법을 모든 버스에 대해 순서대로 진행시킨 후, 정류장에 가장 뒤에 위치한 버스 위치를 출력해주면 된다.

C번 트리나라 관광가이드



트리나라 관광가이드

- 유형: 규칙파악, 트리
- 시간 복잡도 $O(N)$
- **Preorder** 비슷하게 순회하는 문제
- 잘 관찰해보면 어떤 도시가 첫번째로 등장하게 되면, 그 도시 이전에 등장했던 도시가 부모도시일 수 밖에 없음(트리의 구조상)
- 따라서 처음 도착한 도시의 경우 맨 앞이 항상 부모도시. 앞 도시가 없을 경우 루트이므로 -1

D번 영우의 청소



영우의 청소

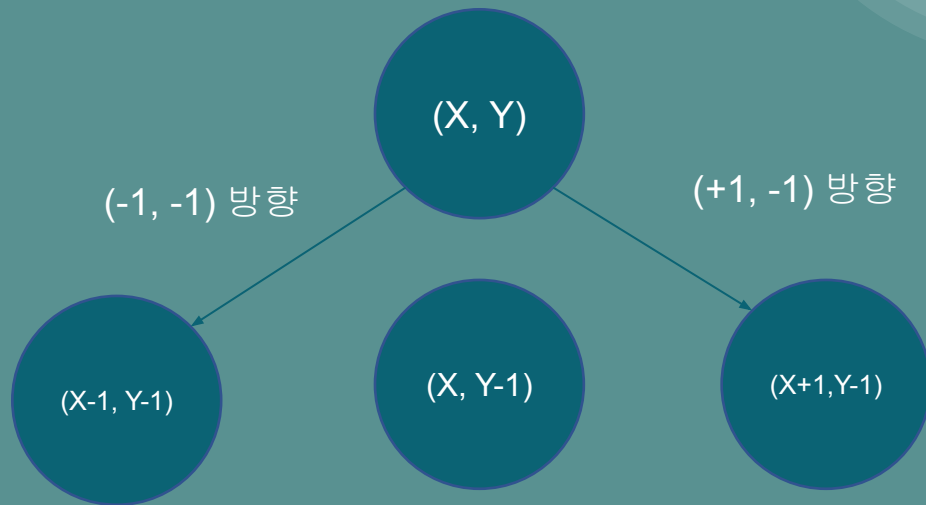
- 유형: 단순 구현, 커팅
- 시간 복잡도 $O(N^3)$
- 곰팡이는 2일에 걸쳐서 대각선 방향으로 3칸씩 이동할 수 있음, 따라서 대략 N 일이 지나면 모든 방으로 퍼진다.
- 곰팡이는 벽에 막혀서 박멸되지 않으면, 2일 이후에 같은 자리에 다시 생긴다.
- 따라서 최대 N 일까지 진행하면서, 검사하는 날짜 t 와 2로나눈 나머지가 같은 날 들을 확인해준다.

우영우
빛
3번



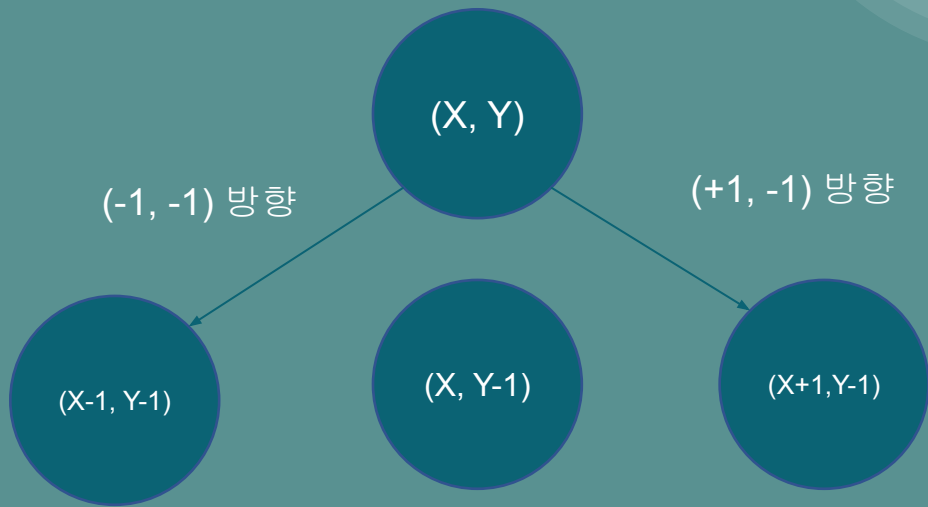
*빛*영*우*

- 유형: DP
- 어떤 지점에 빛이 도달하는 경우는 아래와 같이 밑의 3개 영역에 영향을 받는다.



*빛*영*우*

- $(X, Y-1)$ 이 받는 빛은 무조건적으로 (X, Y) 도 받을 수 밖에 없음
- X, Y 기준으로 $(-1, -1)$ 방향에 위치한 모든 라이트는 $(X, Y-1)$ 에는 영향을 안 주지만 (X, Y) 에는 영향을 줌.
- X, Y 기준으로 $(+1, -1)$ 방향에 위치한 모든 라이트는 $(X, Y-1)$ 에는 영향을 안 주지만 (X, Y) 에는 영향을 줌.



*빛*영*우*

- 따라서 (X, Y) 가 받는 빛의 총량 = $(X, Y-1)$ 이 받는 빛의 총량 + $(-1, -1)$ 방향에 있는 모든 라이트의 수, $(+1, -1)$ 방향에 있는 모든 라이트의 수
- $DY[X][Y][0] = X, Y$ 에서 받는 빛의 총량
- $DY[X][Y][1] = X, Y$ 기준에서 $(-1, -1)$ 방향에 있는 모든 라이트의 수
- $DY[X][Y][2] = X, Y$ 기준에서 $(+1, -1)$ 방향에 있는 모든 라이트의 수
- $LIGHT[X][Y] = X, Y$ 에 존재하는 light의 개수

라고 할 때, 점화식은

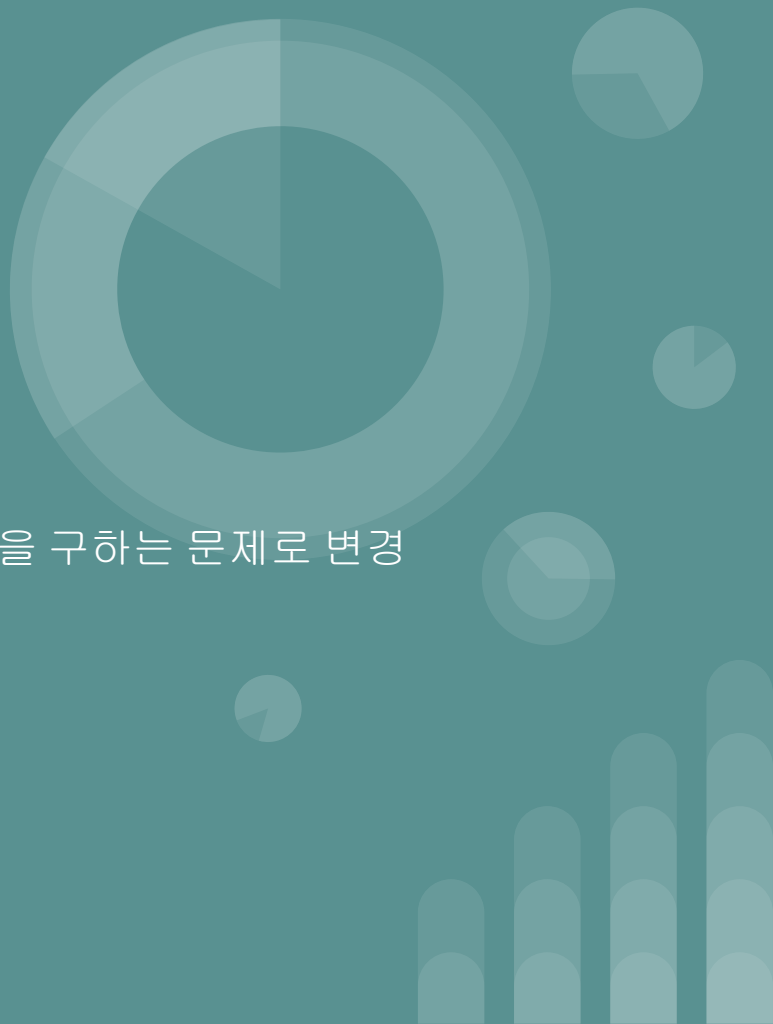
- $DY[X][Y][0] = DY[X][Y-1][0] + DY[X-1][Y-1][1] + DY[X+1][Y-1][2] + LIGHT[X][Y]$
- $DY[X][Y][1] = DY[X-1][Y-1][1] + LIGHT[X][Y]$
- $DY[X][Y][2] = DY[X+1][Y-1][2] + LIGHT[X][Y]$

F번 주말 여행 계획

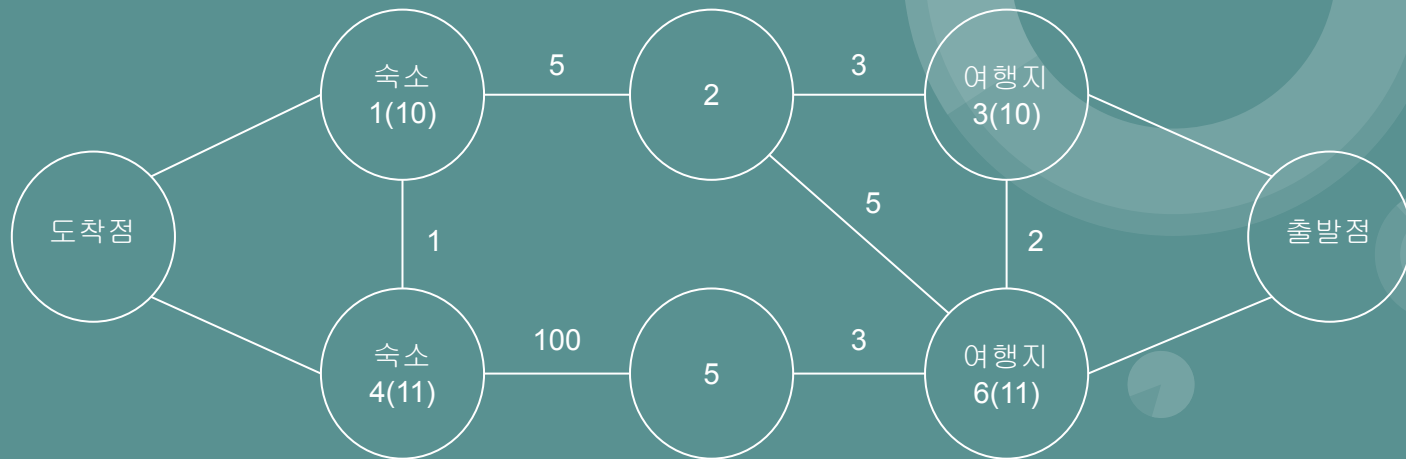


주말 여행 계획

- 유형 : 그래프, 다익스트라
- 시간 복잡도 : $O(N^2)$
- N:N 최단거리 문제
- 거리는 기대치 합에서 빠지게 되므로 사실상 음수
- 거리의 부호를 양수로 생각하면, 최대값에서 최소값을 구하는 문제로 변경
- 즉, 최단거리 문제. 하지만 출발점이 너무 많다.
- 따라서 출발점들을 동시에 처리할 방법이 필요.



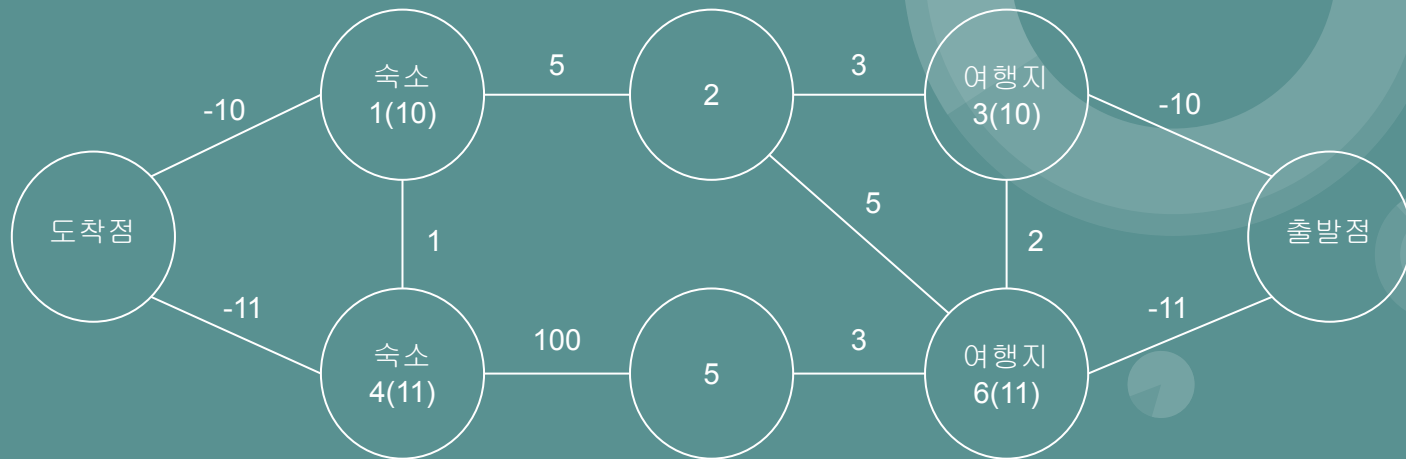
주말 여행 계획



주말 여행 계획

- 다음과 같이 출발점들을 모두 가상의 한 출발점에서 이동하는 것으로 생각한다. (편의상 도착점도 묶는다.)
- 하지만, 해당 그래프에서의 최단거리는 각 여행지와 숙소의 기대치가 고려되지 않는다.
- 따라서 이에 대한 방법으로 여행지와 숙소들이 각각 가상의 점과 연결한 간선에 기대치를 반영한다.
- 이 때, 그래프의 부호는 해당문제와 반대이므로 음수로 반영한다.

주말 여행 계획



주말 여행 계획

- 문제에 명시된 요소를 모두 반영했으니, 해당 그래프에서의 최단거리를 통하여 정답을 찾아낼 수 있다.
- 간선에 음수가 존재하므로 **Bellman Ford algorithm**을 고려해볼 수 있지만, 이는 $O(VE)$ 이므로 이 문제에서는 $O(N^3)$ 으로 시간초과가 발생.
- 이 문제의 그래프 구성에서는 가상의 점으로 들어가고 나가는 간선에만 음수의 가중치를 가진다. 즉, 최단경로의 경우 정확히 2개의 음수간선을 포함한다는 것이 보장된다.
- 따라서 음수간선에 더했을 때 모두 양수로 만들 수 있는 상수 **W**를 정하고, 이를 이용해서 음수간선들을 보정한다. 그 후 그래프의 최단 거리에서 **2W**만큼 뺀 값을 정답으로 사용하면 된다.